

INFORME TÉCNICO EXHAUSTIVO DE

DESARROLLO Y ARQUITECTURA

Implementación Integral de las Historias de Usuario:

HU-RBAC-001: Modelo y Matriz de Permisos por Rol

HU-RBAC-004: Autorización Server-Side y Fail-Closed

Componente	Detalle Oficial
Nombre del Monorepo	The Nexus Battles VI
Scrum Team & Subgrupo	Grupo 4: Cuentas, Cumplimiento y Comercio
Responsable Técnico	Andrés Núñez (AN en el Tablero de Sprint 1)
Microservicio Backend	services/cuentas/ms-identidad (Java 21 LTS, Spring Boot 4.1.1)
Base de Datos	PostgreSQL 16 (contenedor db-ms-identidad, puerto 5433)
Frontend Web	frontend/app-web/src/cuentas/ (Vanilla JS ES2022, sin frameworks)
Seguridad y Sesión	Bearer JWT (HMAC-SHA256) con Versionado (claim ver)
Estándar de Rechazo	RFC 7807 Problem Details (application/problem+json;charset=UTF-8)
Resiliencia y Auditoría	Resilience4j CircuitBreaker hacia ms-cumplimiento + stdout JSON
Ramas de Git Sincronizadas	feat/hub-cuentas-e2e / feature/HU-RBAC-001-004 (commit e0d4e63)
Estado de la Suite de Pruebas	99 pruebas automatizadas pasando (0 fallos, 0 errores, 100% de éxito)

1. Resumen Ejecutivo del Trabajo Realizado

Durante el Sprint 1, el estudiante Andrés Núñez asumió la responsabilidad técnica de diseñar, implementar y asegurar el subsistema completo de autorización y control de acceso para el videojuego *The Nexus Battles VI*. Este trabajo abarcó dos historias de usuario fundamentales y de alto impacto perimetral: **HU-RBAC-001** y **HU-RBAC-004**.

El resultado entregado comprende: (1) un modelo formal de 4 roles y 12 acciones (48 celdas de decisión) con política *Default-Deny*; (2) una directiva reactiva en JavaScript moderno (has-permission) que adapta el DOM del cliente en tiempo real; (3) un interceptor de seguridad perimetral (SecurityInterceptor) en Spring Boot que valida criptográficamente tokens JWT firmados con soporte para revocación instantánea mediante el claim ver; (4) manejo estricto de la política *Fail-Closed* devolviendo respuestas estandarizadas RFC 7807 Problem Details en UTF-8; (5) un cliente asíncrono resiliente de auditoría con CircuitBreaker hacia ms-cumplimiento; (6) un Hub de integración que conecta de punta a punta el flujo de todo el equipo; y (7) una suite de **99 pruebas automatizadas** que verifican rigurosamente que no existen agujeros de seguridad ni posibilidad de bypass.

2. Cumplimiento de las Reglas de Plataforma del Monorepo

El desarrollo se rigió de forma irrestricta por las 12 Reglas de Plataforma y Buenas Prácticas establecidas para el proyecto:

#	Regla de Plataforma	Implementación y Verificación por Andrés Núñez
1	Prefijo /api/v1 en rutas REST	Todos los controladores implementados (RbacController, SecurityInterceptor, AdminCuentasController) operan estrictamente bajo /api/v1/....
2	Frontend sin frameworks (ES2022)	Se utilizó JavaScript Vanilla ES2022 moderno (módulos nativos, manipulación segura del DOM, sin React/Angular/Vue, sin dependencias npm).
3	Un .html + un .js por vista	Arquitectura desacoplada: matriz-permisos.html + matriz-permisos.js, seguridad-servidor.html + seguridad-servidor.js, index.html + index.js. Cero scripts inline.
4	RFC 7807 Problem Details	Respuestas de error con Content-Type: application/problem+json;charset=UTF-8, llaves type, title, status, detail, instance.
5	Logs estructurados JSON a stdout	Interceptor y clientes de auditoría emiten líneas estructuradas AUDIT_TRAIL: {"event": "...", "role": "...", "result": "..."} a stdout.
6	Aislamiento estricto de base de datos	Base de datos exclusiva ms_identidad en puerto PostgreSQL 5433. Ningún acceso cruzado ni esquemas compartidos con otros microservicios.
7	Hilos virtuales Java 21	Habilitado spring.threads.virtual.enabled=true en application.properties para alta concurrencia.
8	Resiliencia con CircuitBreaker	Implementado con Resilience4j hacia ms-cumplimiento y ms-correo para aislamiento de fallos.
9	Tokens de diseño Figma oficiales	Uso exclusivo de variables CSS semánticas (#1c2340, #2244bf, #086b31, #b81a1a), tipografía Inter y cero estilos inline en HTML.
10	Estándar WAI-ARIA y Accesibilidad	Implementación estricta de role="tab", role="tabpanel", aria-selected, aria-controls y estados de foco visibles.
11	Seguridad XSS y DOM seguro	Prohibido el uso de innerHTML con datos no confiables. Renderizadores contruidos con document.createTextNode() y span.textContent.
12	Cero emojis en código e interfaz	Interfaz profesional sobria con insignias semánticas de texto (Habilitado, Restringido, Temporal). Cero iconografía infantil.

3. HU-RBAC-001: Modelo y Matriz de Permisos por Rol

Definición: Como sistema de seguridad de The Nexus Battles VI, requiero gobernar los privilegios de cada cuenta mediante una matriz formal de control de acceso basada en roles, de tal forma que la interfaz de usuario habilite u oculte capacidades de acuerdo con el nivel asignado y el servidor impida cualquier ejecución indebida bajo Default-Deny.

3.1. Criterios de Aceptación y Estado de Cumplimiento

Criterio	Descripción	Implementación Técnica	Estado
CA-1	Cuatro roles oficiales: Jugador, Moderador, Administrador, Super Administrador. Rol desconocido recibe mínimo privilegio.	Modelado en Role.java, persistido vía RolEntity.java y RolSeeder.java. En evaluación, cualquier valor no listado o nulo es denegado.	CUMPLIDO
CA-2	Acciones siguen la matriz canónica (Tabla 24 extendida). Moderador tiene permiso Temporal al suspender usuarios.	RbacMatrixRepository.java y matriz-rbac.js definen PermissionType.TEMPORARY para (MODERADOR, SUSPENDER_USUARIOS).	CUMPLIDO
CA-3	Default-Deny incondicional. Inconsistencias generan registro de advertencia en el sistema.	RbacAuthorizationService.java aplica default-deny y emite advertencia RBAC_INCONSISTENCY_DETECTED ante entradas anómalas.	CUMPLIDO
CA-4	Frontend reactivo que oculta o deshabilita acciones no permitidas en tiempo real.	Directiva has-permission en has-permission.directive.js recorre el DOM y actualiza la visibilidad de forma inmediata.	CUMPLIDO

3.2. Matriz Canónica 12×4 (48 Combinaciones)

La matriz original de la Tabla 24 (10 acciones) fue extendida a 12 acciones para acoplar armoniosamente las historias del equipo: ASIGNAR_ROL (HU-RBAC-003, Edwin) y GESTIONAR_CUENTAS (HU-USR-003, Santiago Sanabria). Las 10 acciones originales preservan con exactitud matemática los permisos del enunciado oficial:

#	Acción de Plataforma	Jugador	Moderador	Administrador	Super Admin	Regla de Negocio / Justificación
1	CREAR_CUENTA_JUGADOR	GRANTED	DENIED	DENIED	DENIED	Auto-registro público. Un admin no se auto-registra.
2	MODIFICAR_PERFIL_PROPIO	GRANTED	GRANTED	GRANTED	GRANTED	Ficha, avatar y biografía propia del usuario.
3	PUBLICAR_COMENTARIOS	GRANTED	GRANTED	GRANTED	GRANTED	Interacción comunitaria en foros y salas de juego.
4	ELIMINAR_COMENTARIO_PROPIO	GRANTED	GRANTED	GRANTED	GRANTED	Derecho de rectificación y supresión de contenido.
5	MODERAR_COMENTARIOS	DENIED	GRANTED	GRANTED	GRANTED	Censura y eliminación de contenido ofensivo de otros.
6	EMITIR_ADVERTENCIAS	DENIED	GRANTED	GRANTED	GRANTED	Notificación disciplinaria preventiva a usuarios.
7	SUSPENDER_USUARIOS	DENIED	TEMPORARY	GRANTED	GRANTED	Moderador suspende temporalmente; Admin inmediato.
8	BANEAR_DEFINITIVAMENTE	DENIED	DENIED	GRANTED	GRANTED	Expulsión irrevocable de la plataforma.
9	CREAR_ADMIN_MODERADOR	DENIED	DENIED	DENIED	GRANTED	Alta de cuentas privilegiadas. Exclusivo Nivel 4.
10	GESTIONAR_PRODUCTOS	DENIED	DENIED	GRANTED	GRANTED	Administración de catálogo de tienda y economía.
11	ASIGNAR_ROL	DENIED	DENIED	DENIED	GRANTED	Cambio de jerarquía de usuarios. Exclusivo Nivel 4.
12	GESTIONAR_CUENTAS	DENIED	DENIED	GRANTED	GRANTED	Búsqueda global y auditoría de perfiles.

T o t	Acciones Autorizadas	4 / 12	7 / 12	10 / 12	11 / 12	Default-Deny: Ningún rol posee 12/12.
-------------	----------------------	--------	--------	---------	---------	---------------------------------------

3.3. Arquitectura Técnica Backend (HU-RBAC-001)

El microservicio ms-identidad organiza la lógica de RBAC siguiendo una arquitectura limpia por capas:

Componente Java	Rol en la Arquitectura	Responsabilidad Concreta
Role.java	Modelo de Dominio	Enum canónico con los 4 roles: JUGADOR, MODERADOR, ADMINISTRADOR, SUPER_ADMINISTRADOR.
Action.java	Modelo de Dominio	Enum con las 12 acciones autorizables del videojuego.
PermissionType.java	Modelo de Dominio	Estados de resolución: GRANTED, TEMPORARY, DENIED.
RoleEntity.java	Persistencia JPA	Mapeo relacional en PostgreSQL (tabla roles con id, nombre, descripcion).
RoleRepository.java	Acceso a Datos	Interfaz Spring Data JPA para consultar roles por nombre (findByNombre).
RoleSeeder.java	Inicialización (Order 1)	CommandLineRunner que siembra los 4 roles en cualquier ambiente al arrancar.
RbacMatrixRepository.java	Repositorio en Memoria	Almacena la matriz inmutable 12x4 en un mapa concurrente para consultas de latencia sub-milisegundo.
RbacAuthorizationService.java	Servicio de Autorización	Ejecuta la evaluación de permisos. Aplica Default-Deny si el rol o la acción son desconocidos o nulos.
RbacController.java	Capa Web REST	Expone endpoints REST: GET /api/v1/rbac/roles, GET /api/v1/rbac/matrix y POST /api/v1/rbac/authorize.

3.4. Implementación Frontend: Directiva Reactiva y Paneles

En el cliente web, la autorización se materializa a través de tres componentes especializados construidos en JavaScript ES2022 estándar:

Archivo Frontend	Propósito	Mecanismo Técnico
has-permission.directive.js	Directiva de Visibilidad	Función applyHasPermissionDirective() que inspecciona elementos con data-has-permission="ACCION" y aplica display: none si el rol activo no tiene permiso.
matriz-rbac.js	Fuente Única de Verdad	Exporta MATRIZ_RBAC y contarAccionesPermitidas(). Es compartida entre el panel de permisos y el Hub para evitar duplicación de código.
matriz-permisos.html / .js	Consola de Inspección RBAC	Panel visual accesible (WAI-ARIA) que detecta automáticamente el backend en puerto 8089 u 8081. Muestra insignias semánticas con variables Figma y selector de 4 roles.

4. HU-RBAC-004: Autorización Server-Side y Fail-Closed

Definición: Como arquitectura de microservicios de The Nexus Battles VI, requiero verificar la credencial de sesión y el permiso de acceso en el servidor para cada petición entrante, independientemente de la interfaz de usuario, aplicando una política estricta Fail-Closed ante anomalías y registrando cualquier intento no autorizado.

4.1. Criterios de Aceptación y Estado de Cumplimiento

Criterio	Descripción	Implementación Técnica	Estado
CA-1	Verificación obligatoria de credencial de sesión y permiso antes de ejecutar la operación.	SecurityInterceptor.java intercepta controladores marcados con @RequirePermission, valida el Bearer JWT y evalúa el permiso en RbacAuthorizationService.	CUMPLIDO
CA-2	Si la autorización no está disponible o falla, denegar incondicionalmente (Fail-Closed).	Ante token nulo, expirado, firma corrupta o error interno, el interceptor aborta y devuelve 403 Forbidden. Nunca deja pasar una petición dudosa.	CUMPLIDO
CA-3	Registro obligatorio de intentos no autorizados con actor, acción, fecha y motivo.	AuditoriaEventClient.java envía evento asíncrono con CircuitBreaker a ms-cumplimiento y emite AUDIT_TRAIL JSON a stdout.	CUMPLIDO
CA-4	Frontend muestra 'no tienes permiso para esta acción' tras 403 en Problem Details.	El servidor emite RFC 7807 UTF-8 con detail oficial. seguridad-servidor.js y http-error.interceptor.js capturan y presentan el error.	CUMPLIDO
CA-5	Mecanismo reutilizable y verificación mediante suite de prueba de penetración/bypass.	Anotación @RequirePermission desacoplada. SecurityBypassTest.java con 7 escenarios de ataque verificados al 100%.	CUMPLIDO

4.2. Flujo de Intercepción Perimetral y Pipeline de Seguridad

Cuando un cliente (legítimo o atacante con Postman/curl) envía una petición hacia un endpoint administrativo, el flujo de ejecución en el servidor sigue un pipeline de defensa en profundidad en 4 etapas secuenciales:

Etapas	Componente Responsable	Acción de Seguridad	Resultado en Falla
1. Detección	Spring MVC HandlerMapping	Detecta si el método o clase tiene @RequirePermission(Action.XYZ).	Si no está protegido, pasa al controlador.
2. Autenticación	SecurityInterceptor + JwtService	Extrae cabecera Authorization: Bearer. Valida firma HMAC-SHA256, expiración y claim ver.	403 Forbidden (CREDENCIAL_MISSING o JWT_INVALID).
3. Autorización	RbacAuthorizationService	Consulta la matriz canónica para el rol extraído del JWT frente a la acción requerida.	403 Forbidden (FORBIDDEN_ROLE por Default-Deny).
4. Auditoría	AuditoriaEventClient	Emite evento asíncrono hacia ms-cumplimiento y registra AUDIT_TRAIL JSON a stdout.	Aislado por CircuitBreaker (nunca tumba el hilo).

4.3. Seguridad Criptográfica de Sesión: JWT v2 y Revocación Instantánea

Para superar la vulnerabilidad de confiar ciegamente en cabeceras de texto plano (como X-User-Role), se implementó una arquitectura criptográfica robusta basada en JSON Web Tokens (JWT):

Propiedad Criptográfica	Implementación en ms-identidad	Beneficio de Seguridad
Algoritmo de Firma	HMAC-SHA256 con clave secreta inyectable app.jwt.clave-secreta.	Impide la falsificación de tokens. Si un atacante altera un carácter del payload, la firma se invalida.
Claims del Token	sub (apodo), rol (rol oficial), ver (versión), iat (emisión), exp (expiración a 24h).	El servidor no almacena estado de sesión (Stateless), facilitando la escalabilidad horizontal.
Control de Revocación (ver)	Cada usuario tiene en PostgreSQL el campo version_token (entero). El JWT viaja con el claim ver.	Revocación inmediata: si un admin degrada a un usuario o suspende su cuenta, version_token se incrementa. Los tokens viejos mueren al instante.
Respaldo Dev-Only	SecurityInterceptor solo acepta X-User-Role si está activo el perfil dev y no hay Bearer token.	Facilita pruebas locales sin comprometer la seguridad en staging o producción.

4.4. Estándar Problem Details RFC 7807 en UTF-8

Ante cualquier rechazo de autorización, SecurityInterceptor no devuelve HTML genérico ni texto plano, sino un documento estructurado según el estándar internacional **RFC 7807 (Problem Details for HTTP APIs)** con codificación UTF-8 explícita:

HTTP/1.1 403 Forbidden
Content-Type: application/problem+json;charset=UTF-8

```
{
  "type": "https://nexusbattles.upb.edu.co/errors/forbidden",
  "title": "Acceso denegado",
  "status": 403,
  "detail": "No tienes permiso para esta acción",
  "instance": "/api/v1/admin/ban"
}
```

4.5. Auditoría Asíncrona y Resiliencia con CircuitBreaker

Todo intento de acceso indebido genera un evento de auditoría emitido por AuditoriaEventClient.java. Para evitar que la caída del microservicio de auditoría (ms-cumplimiento) bloquee el hilo de atención de identidad, se implementó el patrón **CircuitBreaker de Resilience4j** configurado en application.properties:

Propiedad Resilience4j	Valor Configurado	Efecto Operativo
sliding-window-size	10 llamadas	Evalúa la salud de las últimas 10 peticiones hacia ms-cumplimiento.
failure-rate-threshold	50%	Si el 50% de las peticiones fallan, el circuito se abre de inmediato.
wait-duration-in-open-state	30 segundos	Durante 30s no envía tráfico a auditoría, evitando sobrecargar un servicio caído.
fallback-log	stdout estructurado	Si el circuito está abierto, el evento se emite a la consola estándar sin perder la traza.

4.6. Consola de Verificación Server-Side (seguridad-servidor.html / .js)

Para evidenciar de manera irrefutable ante el jurado el funcionamiento del interceptor y la política Fail-Closed, se diseñó una consola técnica de pruebas que interactúa directamente con los endpoints del backend:

Capacidad de la Consola	Detalle Técnico Implementado
Laboratorio de Penetración	Permite simular peticiones con rol arbitrario, sin credenciales, con Bearer JWT válido o con firma corrupta.
Inspección de Red Real	La tabla de encabezados extrae res.headers.get('content-type') y el código HTTP real recibido del servidor.
Pestañas WAI-ARIA Accesibles	Conmutación estricta entre vista de Respuesta JSON RFC 7807 e Inspección de Red mediante teclado (Tab, Enter, Espacio).
Highlighter DOM XSS-Proof	El visor de JSON resalta sintaxis sin vulnerabilidad XSS, creando nodos de texto seguros con document.createTextNode().
Deep-Linking desde el Hub	Acepta parámetros en query string (?rol=JUGADOR&accion;=BANEAR_DEFINITIVAMENTE), precargando la prueba de 1 clic desde el Hub.

5. El Hub Adaptativo E2E: Integración del Flujo del Sprint 1

Para contextualizar el valor de Cuentas, el estudiante diseñó el Hub central (index.html + index.js) estructurando el User Journey del equipo en 4 fases cronológicas:

Fase del Flujo	Integrante Responsable	Historias de Usuario	Rol y Alcance
Fase 1: Onboarding y Acceso	Cristian Camilo Chaparro	HU-AUT-001 HU-AUT-004	Registro con avatar multipart y Login con emisión de Bearer JWT.
Fase 2: Identidad	Santiago Sanabria Uribe	HU-PER-001	Ficha de jugador, edición de biografía y selección de 10 avatares oficiales.
Fase 3: Control de Acceso y Seguridad	Andrés Núñez (AN)	HU-RBAC-001 HU-RBAC-004	Matriz RBAC 12x4, directiva reactiva y defensa Fail-Closed en servidor.
Fase 4: Gobernanza y Sanciones	Santiago Sanabria & Edwin	HU-USR-003 HU-RBAC-003	Búsqueda de usuarios, suspensiones, baneo definitivo y versión de token.

5.1. Usuarios Sembrados para la Demostración (UsuariosDemoSeeder.java)

Se implementó un seeder exclusivo para entornos de desarrollo y prueba (@Profile({"dev", "test"})) con la contraseña unificada **MiClave123!** encriptada en BCrypt:

Rol Oficial	Correo Sembrado	Contraseña	Nivel	Comportamiento en el Hub
Jugador	pruebajugador@test.com	MiClave123!	N1	4/12 acciones habilitadas. Fase 4 de administración bloqueada.
Moderador	pruebamod@test.com	MiClave123!	N2	7/12 acciones. Habilita sanciones temporales comunitarias.
Administrador	pruebaadmin@test.com	MiClave123!	N3	10/12 acciones. Desbloquea Fase 4 y baneo permanente.
Super Admin	pruebasuper@test.com	MiClave123!	N4	11/12 acciones. Acceso total excepto auto-registro por Default-Deny.

6. Trazabilidad y Suite de Pruebas Automatizadas

La suite de pruebas automatizadas en services/cuentas/ms-identidad se ejecuta con Maven Surefire, JUnit 5, Mockito y Spring Boot Test. **Las 99 pruebas del microservicio pasan con un 100% de éxito (0 fallos, 0 errores):**

Clase de Prueba Automatizada	Pruebas	Tipo	Objetivo y Escenarios Críticos Validados	Resultado
SecurityBypassTest	7	Penetración	Pruebas de ataque: atacante anónimo (403), atacante con rol no autorizado (403), token adulterado (403), token revocado por versión (403), admin válido (200), traza de auditoría, RFC 7807.	100% OK
RbacMatrix48CombinationsTest	2	Unitaria	Evaluación exhaustiva de las 48 combinaciones de la matriz canónica y verificación de Default-Deny estricto ante valores nulos.	100% OK
RbacControllerTest	6	Integración	Endpoints REST /api/v1/rbac/roles, /matrix y /authorize. Serialización JSON y manejo de excepciones MethodArgumentNotValid.	100% OK
RoleAssignmentServiceTest	4	Unitaria	Reglas de jerarquía y transición de roles asociadas a la historia HU-RBAC-003.	100% OK
EnvironmentProfileIsolationTest	3	Integración	Aislamiento estricto entre perfiles: asegura que la configuración de desarrollo no se filtre a producción.	100% OK
AuditoriaEventClientTest	1	Resiliencia	Aislamiento mediante CircuitBreaker: ante caída de ms-cumplimiento, el hilo no se bloquea.	100% OK
LoginServiceTest	9	Unitaria	Autenticación con BCrypt, detección de dispositivo nuevo y bloqueo por intentos fallidos.	100% OK
TokenCredencialServiceTest	6	Unitaria	Generación, validación y expiración de tokens temporales de credencial.	100% OK
AdminGestionUsuarioControllerTest	14	Integración	Endpoints administrativos de suspensión, advertencia y baneo de usuarios.	100% OK
Otras suites del microservicio	47	Varios	AdminCuentaService, RegistroService, Perfil, Carga de contexto Spring Boot Application.	100% OK
TOTAL MICROSERVICIO	99	Mixto	Cobertura superior al 80% exigido por JaCoCo. Cero fallos, cero advertencias bloqueantes.	100% ÉXITO

6.1. Evidencia de la Ejecución de Pruebas de Penetración (SecurityBypassTest)

A continuación se presenta el log oficial de ejecución de las pruebas de penetración contra SecurityInterceptor:

```
[INFO] Running com.nexusbattles.ms_identidad.rbac.SecurityBypassTest
21:50:28.425 [main] WARN SecurityInterceptor -- AUDIT_TRAIL: {"event": "SECURITY_BYPASS_ATTEMPT", "username": "jugador_autorizado", "ip": "192.168.1.1"}
21:50:28.438 [main] WARN SecurityInterceptor -- AUDIT_TRAIL: {"event": "SECURITY_BYPASS_ATTEMPT", "username": "unknown", "ip": "192.168.1.1"}
21:50:28.470 [main] WARN SecurityInterceptor -- AUDIT_TRAIL: {"event": "SECURITY_BYPASS_ATTEMPT", "username": "unknown", "ip": "192.168.1.1"}
21:50:29.518 [main] WARN SecurityInterceptor -- AUDIT_TRAIL: {"event": "SECURITY_BYPASS_ATTEMPT", "username": "admin_degradado", "ip": "192.168.1.1"}
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.495 s -- in SecurityBypassTest
```

7. Guion Estratégico de Sustentación en Vivo (3 Minutos)

Para realizar una presentación contundente y ordenada ante los profesores evaluadores, se recomienda seguir rigurosamente este guion cronometrado en 3 actos:

Minuto / Acto	Acción en Pantalla	Guion Verbal Recomendado
Minuto 1: Autenticación y Hub Adaptativo	1. Abrir login.html. 2. Ingresar con pruebajugador@test.com / MiClave123!. 3. Redirige a index.html.	<i>"Buenos días profesores. Iniciamos sesión con una cuenta sembrada de Jugador. Observen cómo el sistema autentica contra PostgreSQL en el microservicio ms-identidad, emite un Bearer JWT criptográfico y nos recibe en el Hub. Dado que nuestro rol es Jugador (Nivel 1), el sistema adapta los módulos: habilita nuestro Perfil y la consulta de Permisos, pero bloquea con borde punteado la Fase 4 de Gestión Administrativa por política RBAC."</i>
Minuto 2: HU-RBAC-001 (Matriz y Directiva)	1. Clic en 'Inspeccionar Matriz RBAC'. 2. Mostrar punto verde de conexión. 3. Cambiar a rol Super Admin. 4. Destacar 11/12.	<i>"Entramos a HU-RBAC-001. Aquí demostramos la matriz canónica 12x4 cargada en caliente desde el backend. Nuestra directiva has-permission en el cliente oculta en el DOM las acciones denegadas. Si selecciono Super Administrador, noten que tiene 11 de 12 acciones y no 12/12. ¿Por qué? Porque CREAR_CUENTA_JUGADOR es un auto-registro público. Concedérsela a un Super Admin violaría el Mínimo Privilegio. Esto demuestra Default-Deny real."</i>
Minuto 3: HU-RBAC-004 (Fail-Closed en Servidor)	1. En el Hub, clic en 'Comprobar 403'. 2. Se abre seguridad-servidor.html por deep-linking. 3. Clic en 'Ejecutar Petición'. 4. Mostrar 403 RFC 7807 y traza.	<i>"Pasamos a HU-RBAC-004. Nunca confiamos en el cliente. Si un atacante intenta saltarse la interfaz con Postman o cURL para invocar el baneo de usuarios, el SecurityInterceptor intercepta la petición en el servidor. Al presionar 'Ejecutar', observen la respuesta real de red: código 403 Forbidden bajo el estándar RFC 7807 en UTF-8 y la emisión asíncrona de la traza de auditoría hacia ms-cumplimiento bajo CircuitBreaker. La política Fail-Closed está 100% verificada."</i>

8. Banco de Respuestas Blindadas ante Preguntas del Jurado

Respuestas técnicas fundamentadas para responder con autoridad ante las preguntas más difíciles del jurado:

Pregunta Probable del Profesor	Respuesta Técnica Blindada
¿Por qué tienen 12 acciones si la Tabla 24 del enunciado solo tenía 10?	<i>"Extendimos la matriz de forma compatible para integrar ASIGNAR_ROL (HU-RBAC-003, Edwin) y GESTIONAR_CUENTAS (HU-USR-003, Santiago Sanabria). Las 10 acciones originales de la Tabla 24 preservan exactamente los mismos permisos definidos en el contrato oficial, garantizando retrocompatibilidad."</i>
¿Qué ocurre si un atacante envía una petición con un rol falso usando cURL o Postman?	<i>"El cliente nunca es de confianza. En el servidor, el SecurityInterceptor valida la firma criptográfica HMAC-SHA256 del Bearer JWT y su versión (ver). Si un atacante envía una cabecera manipulada o un rol falso, el interceptor aborta la ejecución con 403 Forbidden y audita el incidente."</i>
¿Cómo garantizan que un token no se use si un admin le rebajó el rol a ese usuario?	<i>"El token JWT incluye el claim 'ver' (versión de token). Cuando un administrador degrada un rol o suspende una cuenta, incrementa el campo version_token en PostgreSQL. En la siguiente petición, el interceptor detecta que la versión del token no coincide con la de la BD y revoca el acceso de inmediato."</i>
¿Por qué no utilizaron React o Angular para el frontend?	<i>"Cumplimos estrictamente la Regla de Plataforma #2 del monorepo, que exige JavaScript estándar ES2022 (Vanilla JS) sin frameworks para maximizar el rendimiento, garantizar cero dependencias externas y asegurar compatibilidad nativa en navegadores modernos."</i>
¿Cómo manejan la caída del microservicio de auditoría ms-cumplimiento?	<i>"Implementamos el patrón CircuitBreaker de Resilience4j en AuditoriaEventClient. Si ms-cumplimiento no responde o supera el 50% de fallas, el circuito se abre y el interceptor desvía la traza a stdout en formato JSON, garantizando que ms-identidad continúe operando sin degradar la experiencia ni colapsar hilos virtuales."</i>